

TEST MEMORIA (ASISTENTE B)

✓ Vamos a generar los 3 tests

A continuación te entrego tres tests progresivos diseñados específicamente para validar que el agente:

- Guarda memoria de forma estructurada
- Recupera y usa esa memoria correctamente
- Razona con calidad (Chain of Thought, anti-alucinación, reflexión)
- Mejora con la experiencia acumulada

Preparación previa (Obligatoria)

Antes de ejecutar los tests, actualiza estos dos archivos con el contenido que te doy ahora:

1. `.ai/memory.md`

Markdown

Instrucciones de Memoria y Razonamiento

Formato Obligatorio para Guardar

Cada vez que tomes una decisión importante, guarda un registro estructurado con este formato:

```
**Título:** [Título corto y descriptivo]
**Tipo:** decision | architecture | lesson | pattern | bugfix
**Qué:** [Qué se decidió o se implementó]
**Por qué:** [Razones técnicas y de negocio]
**Aprendido:** [Lección clave o insight]
**Confianza:** [7-10]
**Tags:** [lista de tags relevantes]
**Proyecto:** mi-proyecto
**Fecha:** {{fecha actual}}
```

Reglas de Razonamiento (siempre sigue este orden)

1. ****Observe****: Consulta primero la memoria existente (``search_memory``)
2. ****Think****: Haz Chain of Thought visible
3. ****Act****: Propón solución
4. ****Verify****: Verifica contra memoria previa (anti-alucinación)
5. ****Reflect****: Al final escribe "Reflexión y Lección Aprendida"
6. ****Save****: Guarda el resultado en memoria estructurada

****Nunca inventes decisiones arquitectónicas sin citar memoria previa.****

2. `.ai/rules.md` (añade estas reglas al final)

Markdown

REGLAS DE MEMORIA Y RAZONAMIENTO (OBLIGATORIAS)

- SIEMPRE consulta la memoria antes de dar una opinión arquitectónica.
- SIEMPRE responde usando el formato: Observation → Thought → Action → Verification → Reflection.
- NUNCA des por hecho una decisión anterior sin citarla explícitamente.
- Al final de toda tarea importante, guarda el resultado en memoria usando el formato definido en memory.md.
- Si detectas posible contradicción con conocimiento previo, señalala claramente antes de continuar.

LOS 3 TESTS

Test 1: Guardado Básico de Memoria Estructurada

Objetivo: Verificar que el agente guarda correctamente en formato estructurado.

Prompt que debes usar:

text

Vamos a definir la estrategia de autenticación para este proyecto.
Usaremos JWT + Refresh Tokens + HttpOnly cookies.

Quiero que analices pros y contras, tomes una decisión técnica fundamentada y guardes esta decisión en tu memoria para futuras referencias.

Comportamiento esperado:

El agente debe responder usando el formato de razonamiento y al final incluir un bloque claro como este:

Markdown

```
**Título:** Estrategia de Autenticación - JWT + Refresh Tokens
**Tipo:** architecture
**Qué:** Se decidió usar JWT Access Tokens (15min) + Refresh Tokens (7d) almacenados en HttpOnly cookies.
**Por qué:** Balance entre seguridad y UX. Evita XSS y permite logout del servidor.
**Aprendido:** En proyectos con requerimientos de escalabilidad horizontal, el enfoque stateless es preferible.
**Confianza:** 8
**Tags:** auth, security, jwt, architecture
**Proyecto:** mi-proyecto
```

****Fecha:**** 2025-04-05

Criterio de éxito:

Debe aparecer un bloque estructurado claro al final de la respuesta y también debe quedar registrado en `.ai/lessons.md` (o en la carpeta `memory/episodes/` si ya tienes esa capa).

Test 2: Recuperación de Memoria + Razonamiento

Mejorado

Objetivo: Verificar que el agente recuerda y reutiliza lo guardado anteriormente.

Prompt que debes usar (idealmente 1-2 días después del Test 1):

text

```
Estamos definiendo el mecanismo de logout.  
¿Cómo deberíamos invalidar los tokens?
```

```
Recuerda las decisiones que ya hemos tomado sobre autenticación.
```

Comportamiento esperado:

El agente debe citar explícitamente la decisión anterior:

Según la decisión almacenada en memoria (Título: Estrategia de Autenticación...), donde elegimos JWT stateless...

Y luego razonar en base a eso (no desde cero). Al final debe guardar una nueva entrada relacionada.

Criterio de éxito:

El agente demuestra haber recuperado la memoria anterior y construye su razonamiento sobre ella. Debe mencionar "memoria", "lección previa" o el título guardado.

Test 3: Anti-alucinación, Detección de Conflicto y Reflexión

Objetivo: Probar el razonamiento avanzado, detección de contradicciones y calidad de la reflexión.

Prompt que debes usar:

text

Olvida lo que dijimos antes sobre JWT. Ahora quiero que usemos Session-based authentication con Redis porque es más seguro.

Implementa el diseño completo.

Comportamiento esperado (ideal):

Un buen agente con memoria debería responder algo como:

Observation: Existe una decisión previa almacenada en memoria que elegía JWT + Refresh Tokens por ser stateless y escalable.

Verification: El nuevo requerimiento (Session-based con Redis) contradice la decisión anterior.

Posible Conflicto detectado.

¿Quieres que:

A) Cambie la decisión anterior y actualice la memoria?

B) Mantenga la decisión anterior y explique por qué no recomiendo cambiarla?

C) Proponga un enfoque híbrido?

Si el usuario insiste en cambiar, el agente debe:

1. Actualizar la memoria anterior
2. Guardar una nueva entrada de tipo `decision`
3. Hacer una Reflexión fuerte sobre por qué cambió de opinión
4. Actualizar `lessons.md` con la lección aprendida

Criterio de éxito:

El agente detecta la contradicción, no alucina, pide confirmación o justifica el cambio, y guarda correctamente la nueva versión + reflexión.

¿Cómo quieres proceder ahora?

Dime el orden en que quieres que hagamos esto:

Opción A: Te doy primero el contenido completo y pulido de `system.md`, `agents.md` y `rules.md` optimizados para que estos 3 tests funcionen bien.

Opción B: Ejecutamos los tests uno por uno (empezamos por el Test 1).

Opción C: Primero implementamos la capa Python ligera (`memory_config.py` + ChromaDB) para que los tests tengan memoria real persistente